

Reproducing “*No cosmological constraints on dark photon dark matter from resonant conversion*” (arXiv:2510.13956) with Entity Implementation on Lonestar6 and comparison with the published runs

September 21, 2026

Abstract

This note documents a reproduction of the particle-in-cell (PIC) results of Hook, Huang & Shalaby (arXiv:2510.13956v1) using the ENTITY PIC framework in place of the authors’ SHARP code.

Part **I** is an operating manual: the repository layout, a quickstart, a reference for the input deck, job scripts and queue selection, the output format and analysis pipeline, a procedure for sizing a new run, and a consolidated list of the rules and traps that govern all of it. It is reference-ordered and is the only part needed in order to run something. Part **II** covers the implementation: how a problem generator attaches to the framework, built up from a minimal working example; the generator written for this work; a deterministic (“quiet”) particle loader; the four build configurations required on TACC Lonestar6; and a measured performance characterisation across CPU, single-GPU and multi-GPU execution. Part **III** documents every production run, the exact settings used, and a point-by-point comparison with the parameters and results stated in the paper.

Both resonance cases of the paper’s Fig. 2 are reproduced. The Landau–Zener case is not: we measure an electron heating of $\sim 10^3$ where the paper reports a factor below 2, and four candidate explanations have been tested and excluded. Since the resonance cases agree, the disagreement is specific to the swept drive. All performance and physics numbers quoted are measured on Lonestar6 unless explicitly labelled as an estimate; §18 lists the figures from earlier drafts that are superseded, several of which were artefacts of a particle loader that satisfied the paper’s stated initial condition but not its subsequent behaviour.

Contents

1	Scope and conventions	3
I	Working guide	4
2	Repository map	4
3	Quickstart	4
4	The input deck	5
4.1	The [setup] block — the DPDM interface	5
4.2	Settings outside [setup] that are not optional	6
4.3	A complete deck	6

5	Job scripts and the run loop	7
5.1	Template	7
5.2	Choosing a queue	8
5.3	Checkpoints and restarting	8
6	Output and analysis	9
6.1	What a run writes	9
6.2	Deriving the physical quantities	9
6.3	Reading the field dumps	9
7	Sizing a new run	10
8	Rules and traps, consolidated	10
II	Implementation	12
9	Extending Entity: this setup from scratch	12
9.1	Step 1: create the file	12
9.2	Step 2: declare the generator and what it supports	12
9.3	Step 3: read the deck	13
9.4	Step 4: put particles in the box	13
9.5	Step 5: add the dark photon	14
9.6	Step 6: report the drive (optional)	14
9.7	The minimal generator, in full	15
9.8	Step 7: a deck, a build, a run	15
9.9	Summary of the interface	16
10	Problem generator	16
11	Deterministic (“quiet”) particle loading	17
11.1	What the initial condition actually requires	17
11.2	Three noise channels	17
11.3	Construction	17
11.4	Validation: the loader gate	18
11.5	Consequences of the legacy loader	19
12	Building on Lonestar6	19
12.1	Environment and prerequisites	19
12.2	Configure	20
12.3	Compile	20
13	Measured performance	21
13.1	Throughput	21
13.2	Per-step cost doubles after saturation	21
III	Runs, settings, and comparison with the paper	22
14	Parameters taken from the paper source	22
15	Preparatory and calibration runs	22
16	The two resonance cases (paper Fig. 2)	23

17 Landau–Zener runs	24
17.1 The sweep convention	24
17.2 Results, and an unresolved discrepancy	24
17.3 Particle counts, and why the low-amplitude case stays gated	25
18 Superseded results	25
19 Status and remaining work	26

1 Scope and conventions

The paper studies the resonant conversion of dark-photon dark matter (DPDM) into a plasma excitation in the early-universe electron–ion plasma, and shows that *nonlinear* plasma dynamics saturate the conversion far below the linear-theory expectation, weakening the resulting cosmological limits. The evidence is a set of 1D–1V electrostatic PIC simulations.

Stripped of the cosmology, the numerical problem is small and sharply defined:

- 1D electrostatic, periodic box, $N_x = 1000$, $L = 40 c/\omega_p$, $\Delta x = 0.04 c/\omega_p \simeq \lambda_D$;
- electrons and ions with the *real* mass ratio $m_i/m_e = 1836$, both evolved;
- $k_B T_e = k_B T_i = 10^{-3} m_e c^2$;
- the dark photon enters as a *spatially uniform* ($k = 0$), time-oscillating external electric field $E_{\text{ext}}(t) = A_0 \cos(\omega t)$ acting on the particle Lorentz force *only* — it is never a source term in Maxwell’s equations. All feedback is through the deposited current.

Code units. Throughout, $\omega_{pe} = 1$, $c = 1$, $n_e = n_0$, $m_e = 1$. Energies are in $n_e m_e c^2$, times in ω_p^{-1} , wavenumbers in ω_p/c . The drive amplitude in code units equals the paper’s: $A_0 = v_q^D/c = (v_q^D/v_{\text{th}}^e)\sqrt{10^{-3}}$.

Measured versus estimated. Every performance and physics number here is measured on Lonestar6 unless explicitly labelled an estimate. Section 18 lists the few figures from earlier drafts that are superseded, so that they are recognisable if they turn up in older scripts or notes.

Part I

Working guide

This part is reference-ordered and self-contained: it is what to read in order to run something. Part II explains why the code is built the way it is, and Part III records what the production runs produced. Neither is needed to operate the code.

2 Repository map

Everything specific to this work lives in two directories of the ENTITY tree.

Path (under repository root)	Contents
<i>pgens/dpdm/</i> — <i>problem generator, decks, job scripts</i> <code>pgen.hpp</code> <code>resonance.toml</code> <code>landau_zener.toml</code> <code>run_paper_fast.sh</code> <code>run_paper_slow.sh</code> <code>run_paper_slow_resume.sh</code> <code>run_lz1p0.sh</code>, <code>run_lz0p1.sh</code> <code>run_gate_loader.sh</code> <code>run_paper_fast_shear.sh</code>, <code>run_paper_slow_shear.sh</code>, <code>run_lz1p0_shear.sh</code> <code>run_lz1p0_naive.sh</code> <code>run_lz_heating.sh</code> <code>validate_quiet.sh</code>, <code>check_quiet.py</code> <code>validate_mpi.sh</code> <code>bench_slow.sh</code>, <code>bench_gpu.sh</code>, <code>bench_multigpu.sh</code> <code>diag_slow.sh</code>, <code>diag_growth.sh</code>, <code>affinity_test.sh</code>	The problem generator: external drive, diagnostic. The only C++ in this work (Sweep-frequency family. Reference deck, fixed-frequency family. Reference deck, swept-frequency family. Production: $v_q^D/v_{th}^e = 3 \times 10^{-2}$, $N_{pc} = 2$. Production: $v_q^D/v_{th}^e = 1 \times 10^{-3}$, $N_{pc} = 2$. The same run restarted from checkpoint. Landau–Zener production at $v_q^D/v_{th}^e = 1$. The loader gate: three undriven runs, one the paper’s Fig. A00 (§11.4). The three production re-runs with the correct decks; new output roots. Landau–Zener with <code>sweep_phase = "naive"</code>. Undriven numerical-heating gate at Landau–Zener. Quiet-start validation gates (§11.4). Confirms results are independent of MPI. Throughput benchmarks: CPU/MPI, 1 CPU. Diagnostics for performance and thread-pinning.
<i>pgens/dpdm/paper/</i> — <i>analysis pipeline for the paper-faithful runs</i> <code>extract_paper.py</code>	Reduces <code>fast</code>, <code>slow</code>, <code>lz1p0</code>, <code>heat</code> and <code>heat_shear</code> to <code>.npz</code> (§6). The six plots those runs support, and what each plot shows.
<i>pgens/dpdm/run34/</i> — <i>the resonance-ladder survey and its analysis pipeline</i> <code>run34.sh</code> <code>extract_run34.py</code> <code>PLOTS.md</code> <code>run34_plots.ipynb</code>	Eight-run ladder in v_q^D/v_{th}^e plus two conversion runs. Reduces ADIOS2 output to <code>.npz</code> (§6). Which four plots to make and what each plot shows. The plots, from the <code>.npz</code> files.
<i>dpdm_doc/</i> — <i>this document</i> <code>main.tex</code>, <code>Makefile</code>	Source and build (<code>make</code> → <code>main.pdf</code>).

Build directories (`build-dpdm*`) are deliberately not version controlled; see §12. Run output goes to `$SCRATCH`, which is **purged on a rolling basis** — copy anything you intend to keep to `$WORK`.

3 Quickstart

End to end, on a login node unless stated otherwise:

1. **Build once.** Follow §12. For a first run the CPU OpenMP configuration (`build-dpdm`) is enough; the CUDA build (`build-dpdm-cuda`) is what production uses.

2. **Verify the build** — configure options fail silently here:

```
grep -E "^Kokkos_ENABLE_(OPENMP|SERIAL|CUDA):BOOL" build-*/CMakeCache.txt
```

3. **Copy a job script** from `pgens/dpdm/` that matches the shape of what you want — they are self-contained and embed their own deck — and edit the deck inside it (§4).
4. **Size the run:** particle count, GPU count and walltime follow a fixed procedure (§7). The GPU count in particular cannot be changed after the first submission if the run will ever restart.
5. **Submit:** `sbatch run_my_case.sh`. Jobs under 2 h belong in development (§5.2).
6. **Analyse:** the stats CSV answers most questions directly; field and particle dumps go through `extract_run34.py` (§6).

4 The input deck

4.1 The [setup] block — the DPDM interface

These nine keys are the whole problem-specific interface; everything else in the deck is stock ENTITY. Defaults are those in `pgen.hpp`.

Key	Default	Meaning
<code>drive</code>	<code>"resonance"</code>	<code>"resonance"</code> (fixed ω) or <code>"landau_zener"</code> (swept).
<code>loading</code>	<code>"random"</code>	<code>"quiet"</code> is the paper's IC: lattice positions, sampled velocities, relaxing to the Poisson floor. <code>"quiet_lattice"</code> is the legacy permanently-ordered loader, kept only to reproduce superseded runs — it does <i>not</i> reproduce the paper (§11.5). <code>"random"</code> is the stock injector and has $\epsilon_{\text{noise}}(0) \neq 0$. Set this explicitly.
<code>one_v</code>	<code>true</code>	Sets $u_{x2} = u_{x3} = 0$, matching SHARP and the paper's 1V thermal speed. Leave on for paper comparison.
<code>A0</code>	<code>0.0</code>	Drive amplitude in code units, $A_0 = v_q^D/c = (v_q^D/v_{\text{th}}^e)\sqrt{10^{-3}}$.
<code>omega</code>	<code>1.0</code>	Drive frequency, <code>drive="resonance"</code> only.
<code>sweep_phase</code>	<code>"integrated"</code>	<code>"integrated"</code> gives instantaneous frequency $\omega_0 + \dot{\omega}t$ and is correct; <code>"naive"</code> reproduces the literal reading of the paper's notation and sweeps at twice the rate (§17.1). <code>landau_zener</code> only.
<code>omega0</code>	<code>0.0</code>	Initial frequency, <code>drive="landau_zener"</code> only.
<code>domega_dt</code>	<code>0.0</code>	Sweep rate, <code>drive="landau_zener"</code> only.
<code>densities</code>	<code>[2.0]</code>	One entry, per species <i>pair</i> . <code>[2.0]</code> with <code>skindepth0 = 1</code> gives $n_e = n_0$, hence $\omega_{pe} = 1$.
<code>temperatures</code>	<code>—</code>	One entry per species, in $m_0 c^2$. The injector divides by species mass, so <code>[1.0e-3, 1.0e-3]</code> is $10^{-3} m_e c^2$ for both.

The two drive families are mutually exclusive in their parameters: `omega` is ignored when sweeping, and `omega0/domega_dt` are ignored when not. An unrecognised `drive` or `loading` aborts at startup with a readable message rather than running the wrong physics.

4.2 Settings outside [setup] that are not optional

- `particles.ppc0` must be written as a **float**. `ppc0 = 2000` aborts with `toml::value::as_floating(): bad_cast`; write `2000.0`.
- `maxnpart` per species must exceed $\text{ppc0} \times \text{densities}[0] / 2 \times N_x$, with headroom.
- A `[diagnostics]` block is required in practice. Without it, `<name>.log` is a kernel trace at $\sim 4\text{kB}$ *per step* and `<name>.out` is a progress bar redrawn every step, both written synchronously to the parallel filesystem. Use `log_level = "WARNING"`, `interval = 1000`, `colored_stdout = false`.
- `[output.spectra]` defaults to *on*; set `enable = false`.
- `[output.stats]` `interval_time` must be ≥ 0.5 , and preferably ≥ 1.0 . Per-species `T00.1/T00.2` force a moment reduction over every particle; at 0.1 this doubles wall-clock.
- Stats quantity names are literal: "E^2", "T00.1", "T00.2", "J.E". "JdotE" aborts. "T00" alone gives the species sum.
- `algorithms.current_filters = 0`. The saturation physics lives at high k ; filtering removes exactly the modes of interest.

4.3 A complete deck

The fast resonance case in full. Every other run in this work is this file with `ppc0`, `A0`, `runtime` and the `[setup]` drive keys changed.

```
[simulation]
  name = "fast"
  engine = "srpic"
  runtime = 10000.0          # omega_p t
[grid]
  resolution = [1000]        # Nx
  extent = [[0.0, 40.0]]     # L = 40 c/omega_p, so dx = 0.04 ~ lambda_D
[grid.metric]
  metric = "minkowski"
[grid.boundaries]
  fields = [["PERIODIC"]]
  particles = [["PERIODIC"]]
[scales]
  skindpth0 = 1.0            # with densities=[2.0] this sets omega_pe = 1
  larmor0 = 1.0              # unused: no magnetic field
[algorithms]
  current_filters = 0         # KEEP AT 0 -- see above
[algorithms.timestep]
  CFL = 0.5                  # dt = 0.02 / omega_p
[particles]
  ppc0 = 200.0               # N_pc per cell per species; Np = N_pc*Nx = 2e5
  [[particles.species]]
    label = "e-"
    mass = 1.0
    charge = -1.0
    maxnpart = 2.5e5
    pusher = "Vay"
  [[particles.species]]
    label = "i+"
    mass = 1836.0            # real mass ratio; ions are evolved
    charge = 1.0
    maxnpart = 2.5e5
    pusher = "Vay"
[setup]
  densities = [2.0]
  temperatures = [1.0e-3, 1.0e-3]
  drive = "resonance"
  loading = "quiet"          # NOT the default -- must be stated.
                              # "quiet" = lattice positions + sampled velocities.
```

```

# "quiet_lattice" is the legacy loader; it never
# decoheres and does not reproduce the paper.

one_v = true
A0 = 9.48683e-4      #  $v_q^D/v_{th_e} = 3e-2$ 
omega = 1.0

[diagnostics]
interval = 1000
log_level = "WARNING"
colored_stdout = false

[output]
interval_time = 10.0      # field/particle dumps; 1000 of them here
[output.fields]
quantities = ["E", "N_1", "N_2", "Rho", "J", "T00_1", "T00_2"]
[output.particles]
species = [1, 2]
stride = 20
[output.stats]
enable = true
interval_time = 0.5
quantities = ["E^2", "T00_1", "T00_2", "J.E"]
custom = ["e_ext"]      # the drive itself, for phase checks
[output.spectra]
enable = false
[checkpoint]
interval_time = 2500.0
keep = 2

```

For the swept family, replace the drive keys:

```

drive = "landau_zener"
A0 = 3.16228e-3      #  $v_q^D/v_{th_e} = 0.1$ 
omega0 = 0.8         # start below resonance
domega_dt = 2.0e-5   # 0.1  $\omega_p$  over  $\omega_p$   $t = 5000$ 
sweep_phase = "integrated" # crossing at  $\omega_p$   $t = 1e4$ ;
                        # "naive" doubles the rate -> crossing at 5000

```

5 Job scripts and the run loop

5.1 Template

Job scripts in `pgens/dpdm/` are self-contained: they carry their own deck as a heredoc, so a script plus its log is a complete record of what was run. The skeleton:

```

#!/bin/bash
#SBATCH -J dpdm_fast
#SBATCH -o sbatch_logs/out/fast.o%j
#SBATCH -e sbatch_logs/err/fast.e%j
#SBATCH -p development      # see queue selection below
#SBATCH -N 1
#SBATCH -n 1
#SBATCH -t 01:30:00
#SBATCH -A PHY23028
#SBATCH --mail-type=all
#SBATCH --mail-user=<you>@utexas.edu

module load gcc/13.2.0      # REQUIRED at run time, not only at build time

export OMP_NUM_THREADS=112  # CPU builds only; omit for CUDA
export OMP_PROC_BIND=close
export OMP_PLACES=cores

ROOT=$SCRATCH/dpdm/paper/fast

```

```

EXE=<repo>/build-dpdm/src/entity.xc
mkdir -p $ROOT/fast                # pre-create <simulation.name>/ -- see below

cat > $ROOT/fast.toml <<EOF
... deck ...
EOF

cd $ROOT && $EXE -input fast.toml > fast.stdout 2>&1
echo "exit=$? $(date -Is)"

```

Three things in that skeleton are load-bearing:

- `module load gcc/13.2.0` inside the job. The binary links GCC 13's `libstdc++`; the default environment is `gcc/9.4.0` and the executable dies with `GLIBCXX_3.4.31 not found` *before* reading its input.
- `mkdir -p $ROOT/<simulation.name>`. Under MPI every rank calls `mkdir` on that directory and the losers abort with `filesystem error: cannot create directory: File exists, exit 255`, within seconds. A job that "finished" in four seconds is this, not speed.
- `sbatch_logs/out/` and `sbatch_logs/err/` must exist before submission or the job is rejected.

MPI runs replace the last line with `ibrun` and set `-n` to the rank count; single-GPU runs need no launcher and no `OMP_*` variables.

5.2 Choosing a queue

Partition	MinNode	MaxNode	MaxWall	MaxNodePU	MaxJobsPU
development	1	8	02:00:00	8	1
normal	1	64	2-00:00:00	64	20
gpu-a100	1	8	2-00:00:00	12	8
gpu-a100-dev	1	2	02:00:00	2	1
gpu-a100-small	1	1	2-00:00:00	3	3

- **Under 2 h and ≤ 8 nodes: use development.** Measured turnaround there was 9 s and 70 s for jobs that never started at all in normal. `MaxJobsPU = 1` only means short jobs run back-to-back, which still beats normal priority.
- **Single-GPU production: gpu-a100-small.** One A100 per node, billing weight 32, versus three per node at weight 128 on `gpu-a100`.
- **Prefer fewer nodes than the scaling curve suggests.** On a busy machine queue wait swamps compute: multi-node GPU jobs requesting 2 and 4 nodes sat unscheduled for 30 h and were cancelled, while single-node jobs started within minutes. A 12-GPU configuration with a 6 h runtime is worthless if it waits 30 h for contiguous nodes.

5.3 Checkpoints and restarting

Checkpoints are controlled by `[checkpoint] interval_time` with `keep` retained, and written to `<name>.ckpt/` as `step-NNNNNNNN.bp` plus a meta-`NNNNNNNN.toml`. A run resumes with

```
cd $ROOT && $EXE -input fast.toml -resume > fast.resume.stdout 2>&1
```

(aliases: `-continue`, `-restart`, `-checkpoint`). ENTITY selects the latest checkpoint automatically. Increase runtime in the deck if the original value has already been reached.

The domain count is locked by the checkpoint format. An N -domain checkpoint can only be resumed by N domains: a single-GPU checkpoint cannot be resumed by a 3-GPU binary. Decide the rank/GPU count before the *first* submission of any run that might need to restart — there is no way to change it later without starting over.

6 Output and analysis

6.1 What a run writes

Output lands in `<rundir>/<simulation.name>/`:

File	Contents
<code><name>_stats.csv</code>	Scalar time series, one row per <code>[output.stats] interval_time</code> . Answers most questions on its own.
<code>fields/*.bp</code>	ADIOS2 BP5 field dumps at <code>[output] interval_time</code> .
<code>particles/*.bp</code>	Particle dumps, decimated by <code>[output.particles] stride</code> .
<code><name>.log, <name>.out, <name>.info, <name>.err</code>	Logs; see §4.2 before the first production run.
<code><name>.ckpt/</code>	Checkpoints (sibling of <code><name>/</code> , not inside it).

The stats CSV columns, in order, are `step`, `time`, `E1^2`, `E2^2`, `E3^2`, `T00_1`, `T00_2`, `J.E`, and any custom entries (here `e_ext`). In 1D electrostatic runs `E2^2` and `E3^2` are zero by construction.

6.2 Deriving the physical quantities

The conversions below are the ones the plots need, and two of them are easy to get wrong in a way that changes conclusions by a factor of two or three.

- $\epsilon_E = \frac{1}{2} \langle E_x^2 \rangle$ from `E1^2`.
- $\Delta KE_e = T00_1(t) - T00_1(0)$, and likewise for the ions. Taking the difference is what makes this safe under ENTITY's 3V convention: with no transverse forces the perpendicular energy is constant and cancels.
- **The linear-theory line is $\epsilon_E + \Delta KE_e = CA_0^2(\omega_p t)^2$ with $C = 1/8$, compared against the *sum*, never ϵ_E alone.** A resonantly driven Langmuir wave equipartitions, so the field holds half the wave energy on cycle average and swings between 3% and 85% of it instantaneously at $2\omega_p$.
- **The electron thermal reference is the 1V value** $\epsilon_{th} = \frac{1}{2} n_e v_{th,e}^2 = 5 \times 10^{-4}$, not `T00_1 - 1 =  $1.5 \times 10^{-3}$` , which is the 3V rest-frame value.
- `Rho` in the field output is *mass* density (`1 + 1836 = 1837` in code units); the charge density is the separate `Charge` field.
- `CustomStat` receives `time` one step behind the `time` column of the same row (`engine.hpp` passes `time - dt` to the writer). This affects `e_ext` bookkeeping only; the pusher's centring is correct.

6.3 Reading the field dumps

Field and particle output is ADIOS2 BP5 and is read with `nt2py`:

```
pip install nt2py          # brings adios2 + xarray + dask
```

```
import nt2
data = nt2.Data("<rundir>/<name>/<name>")    # note: name repeated
```

`pgens/dpdm/run34/extract_run34.py` is the worked pipeline: it reduces a full survey (~ 2.7 GB of `.bp`) to ~ 65 MB of `.npz` — `<tag>_stats.npz`, `<tag>_fields.npz`, `<tag>_temps.npz`, plus `summary.npz` and `meta.json` — so that plotting needs only `numpy`. Set `run34_dir` at the top of the file and run it once. This is the intended route for analysing on a laptop: reduce on the cluster, download the `.npz`.

`run34/PLOTS.md` states which four plots matter and what each one claims: the energy budget (paper Fig. 2), the $E(k, t)$ and $n(k, t)$ spectra that show *why* it saturates, the N_p convergence gate, and the fast/slow regime transition. `run34.plots.ipynb` produces them from the `.npz`.

`pgens/dpdm/paper/extract_paper.py` is the same pipeline for the production runs, which `run34` cannot substitute for: `run34` is a random-start survey at a uniform $N_{\text{pc}} = 2000$ out to $\omega_p t = 3000$, whereas these are quiet-start, at the paper’s own N_{pc} , and include the swept-frequency family. It covers `fast`, `slow`, `lz1p0`, `heat` and the `undriven_q/undriven_r` pair, adding (x, u_x) phase-space histograms to the products, and reduces ~ 1.9 GB of `.bp` to ~ 50 MB. Stage the runs without their checkpoint directories first — `slow` alone carries 75 GB of checkpoints beside 483 MB of science. `paper/PLOTS.md` states the six plots.

7 Sizing a new run

Given a dark-photon amplitude v_q^D/v_{th}^e , in order:

1. **Amplitude.** $A_0 = (v_q^D/v_{\text{th}}^e)\sqrt{10^{-3}}$.
2. **Particle count.** Use the paper’s shot-noise floor $\epsilon_{\text{noise}} = L^2/(12N_p N_x)$, in which N_p is **the total macroparticle count**, $N_p = N_{\text{pc}} N_x$ — not the per-cell count. So evaluated, it agrees with measurement at a flat factor 3.1 across three decades of N_{pc} , and is the right tool for this job. In code units the measured floor is $\epsilon_{\text{noise}} = 2.06 \times 10^{-4}/N_{\text{pc}}$.
 - *Resonant drive:* the signal grows as t^2 , so it need only outgrow the noise before saturation.
 - *Swept drive:* the pre-resonance response is a *bounded* forced oscillation, $A_0/(1 - \omega^2/\omega_p^2) = 2.78A_0$ at $\omega = 0.8\omega_p$, so there is no time at which signal outgrows noise. Use the paper’s criterion $\bar{t}_{\text{noise}} = L/(A_0\sqrt{3N_p N_x/2}) \leq 0.02$, giving $N_{\text{pc}} = 2.7 \times 10^3$ at $v_q^D/v_{\text{th}}^e = 1.0$ and 2.7×10^5 at 0.1 .

A quiet start does not reduce N_{pc} ; see §11.5.

3. **Memory, hence GPU count.** At $N_{\text{pc}} = 2.7 \times 10^5$ a run holds 5.3×10^8 particles and ≈ 31 GB — tight on one 40 GB A100, comfortable at ≈ 10.5 GB per GPU across three. Because the checkpoint domain count is locked (§5.3), settle this before submitting.
4. **Walltime.** Take the per-step cost from Table 4, then **double it**, or benchmark across the saturation transition. Per-step cost roughly doubles once the plasma becomes turbulent ($265 \rightarrow 516$ ms on one A100 at $N_{\text{pc}} = 2 \times 10^5$); a benchmark run entirely inside the laminar phase measures the wrong number. $\text{Steps} = \text{runtime}/0.02$.
5. **Queue.** §5.2. If the estimate exceeds the 48 h cap, plan the restart chain from the start.

8 Rules and traps, consolidated

Everything below is a standing rule. Each is stated once here; the evidence for it is in Parts II and III.

Building.

1. Configure and compile on a **login node**: `FetchContent` needs outbound network and compute nodes have none.
2. `module load gcc/13.2.0` at build time *and* in every job script.
3. Pass `-D Kokkos_ENABLE_OPENMP=ON -D Kokkos_ENABLE_SERIAL=OFF` explicitly. `CMAKE_IGNORE_PATH`, which is required to make the ADIOS2 fetch work, also blocks the repository’s Kokkos settings, and a freshly fetched Kokkos defaults to Serial only — a build that is externally indistinguishable and $\sim 112\times$ slower. Neither SLURM’s `-c` default nor `ibrun` binding causes this; do not go looking there.
4. Always verify the cache after configuring (§3, step 2).

Decks.

5. `ppc0` must be a TOML float.
6. `loading = "quiet"` must be stated; the default is "random". Do not use "quiet_lattice" for new work — it never decoheres (§11.5).
7. Include `[diagnostics]`, disable `[output.spectra]`, keep `[output.stats] interval_time ≥ 1.0`, keep `current_filters = 0`.

Running.

8. Pre-create `<rundir>/<simulation.name>/` before an MPI run.
9. Fix the rank/GPU count before the first submission: checkpoints lock it.
10. Budget a factor two on per-step cost across saturation.
11. Jobs under 2h go to `development`; prefer fewer nodes.
12. `blocking_timers = true` is the first move on any performance surprise. It prints a per-kernel breakdown in ns per particle-step and settles in seconds questions that speculation gets wrong.

Analysis.

13. Compare linear theory against $\epsilon_E + \Delta K E_e$, not ϵ_E .
14. Use the 1V thermal reference 5×10^{-4} .
15. N_p in the shot-noise formula is the total macroparticle count.
16. `Rho` is mass density; charge density is `Charge`.
17. $E_x(t=0) = 0$ proves nothing about the loading — ENTITY initialises E to zero identically and every loader co-locates the species. The discriminator is the total density $n_e + n_i$.
18. A perfectly ordered lattice is *not* the goal. The paper's initial condition is $\epsilon_{\text{noise}}(0) = 0$ followed by relaxation to the Poisson floor within a few ω_p^{-1} . Validate against both halves: a loader that stays ordered seeds daughter modes from round-off and delays saturation (§11.4).
19. ENTITY's first output row is step 1, not $t = 0$, so no dump shows the initial condition directly.

Part II

Implementation

9 Extending Entity: this setup from scratch

This section assumes no prior knowledge of ENTITY and builds the DPDM resonance setup from nothing. It is the orientation for the two sections that follow: §10 describes what the production generator *does*, §12 how to compile it; this one explains what a problem generator *is*, how the pieces attach to the framework, and what the irreducible minimum looks like.

The mental model. ENTITY’s core is a fixed engine: Maxwell solver, particle pusher, current deposit, filtering, I/O, MPI decomposition. None of it is edited to add a new physics problem. Instead a *problem generator* (“pgen”) — one C++ header — is compiled into the engine, and it answers three questions:

1. What particles exist at $t = 0$, and where?
2. What extra forces act on them, beyond the self-consistent fields?
3. What extra numbers should be written to the diagnostics?

Everything specific to this work is an answer to one of those three. The important structural fact is that a binary contains **exactly one** problem generator, chosen at configure time, not at run time — so “build the resonance case” and “build the Landau–Zener case” are the same executable, while “build the reconnection case” is a different one.

9.1 Step 1: create the file

```
pgens/dpdm/pgen.hpp
```

That path is the entire registration procedure. CMake globs `pgens/**/pgen.hpp` at configure time — see `get_available_pgens` in `cmake/config.cmake` — and `-D pgen=dpdm` selects the directory whose name matches. The selected header is then attached as an INTERFACE library linked against the core targets, by `set_problem_generator` in the same file. Three consequences worth stating plainly, because each removes a step one would otherwise look for:

- there is no registry, enum or dispatch table in the core to add a name to;
- there is no `CMakeLists.txt` inside the pgen directory;
- the *directory name* is the value of the `pgen` option — there is no separate identifier declared inside the file.

Anything else in the directory (decks, plotting scripts, job scripts) is ignored by the build and is there purely for the humans.

9.2 Step 2: declare the generator and what it supports

The header defines one class template in namespace `user`:

```
template <SimEngine::type S, class M>
struct PGen {
    static constexpr auto D { M::Dim };
    static constexpr auto engines    = traits::pgen::compatible_with<SimEngine::SRPIC>{};
    static constexpr auto metrics    = traits::pgen::compatible_with<Metric::Minkowski>{};
    static constexpr auto dimensions = traits::pgen::compatible_with<Dim::_1D>{};
    ...
};
```

`S` is the physics engine and `M` the metric; the core instantiates the template for whatever the deck asks for. The three `compatible_with` traits are declarations of intent, checked at compile time: they say this generator is written for special-relativistic PIC on a flat Minkowski grid in

one dimension. A deck asking for anything else is rejected rather than silently producing wrong physics. Declaring more than one value is allowed — the production generator declares `Dim::_1D`, `Dim::_2D`, `Dim::_3D` — but every declared combination has to actually work.

9.3 Step 3: read the deck

The constructor receives the parsed input file and the global domain:

```
PGen(const SimulationParams& p, const Metadomain<S, M>& global_domain)
: params { p }
, A0 { p.template get<real_t>("setup.A0", ZERO) }
, omega { p.template get<real_t>("setup.omega", ONE) }
, densities { p.template get<prmvect_t>("setup.densities", prmvect_t{TWO}) }
, temperatures { p.template get<prmvect_t>("setup.temperatures", prmvect_t{ZERO,ZERO}) }
{ /* validation goes here */ }
```

This is where the `[setup]` block of §4 comes from, and the relationship is worth being explicit about: `[setup]` is a **free-form namespace owned entirely by the problem generator**. The core never inspects it. There is no schema, no list of legal keys, and nothing outside this one header knows that `setup.drive` or `setup.loading` exist. Each option therefore has exactly three touch points in `pgen.hpp` — a member declaration, a `get` call with its default, and the site that consumes it:

Key	declared	read (default)	consumed
<code>setup.drive</code>	:301	:310 ("resonance")	<code>drive_phase()</code> , :382
<code>setup.loading</code>	:301	:311 ("random")	<code>InitPrtls()</code> , :433
<code>setup.sweep-phase</code>	:303	:319 ("integrated")	<code>drive_phase()</code> , :383
<code>setup.A0</code>	:304	:321 (0)	<code>ext_efield()</code> , :393

Table 1: Line numbers in `pgens/dpdm/pgen.hpp`. Adding an option means editing these three places and nothing else.

A trap that follows directly from this design. The second argument to `get` is a fallback, so an absent key is legal — which means a *misspelled key silently takes the default*. Writing `loading = "quiet"` runs the random loader, with no error and no warning, and the run looks entirely normal. Only the *value* of a key is ever validated, never its name. Validation is manual, in the constructor body:

```
raise::ErrorIf(loading != "random" and loading != "quiet" and
               loading != "quiet_lattice",
               "setup.loading must be 'random', 'quiet', or 'quiet_lattice'", HERE);
```

The same mechanism is used for cross-checks the deck cannot express on its own: that exactly two species are defined, and that their charges are equal and opposite.

9.4 Step 4: put particles in the box

```
void InitPrtls(Domain<S, M>& domain) {
    arch::InjectUniformMaxwellians<S, M>(params, domain, densities[0],
                                          { temperatures[0], temperatures[1] }, { 1, 2 });
}
```

One call. `arch::` is ENTITY’s library of ready-made “archetypes”; this one fills the domain with a co-located electron–ion pair at the requested density and temperatures, drawing positions uniformly at random. For a great many problems this line is the whole of particle initialisation, and for the minimal resonance setup it is sufficient.

It is also the exact point at which this work had to leave the beaten path. The injector exposes two customisation seams — the velocity distribution (`EnrgDistClass`) and a density profile (`SpatialDistClass`, which sets *how many* particles land in a cell) — but the sub-cell *position* is hard-coded as a random draw inside the kernel, with no seam at all. A quiet start is by definition a statement about positions, so it cannot be expressed through the archetypes and has to be a hand-written kernel. That is the subject of §11, and it is why `setup.loading` exists at all.

9.5 Step 5: add the dark photon

The drive is an external electric field that acts on the Lorentz force only; it is never a source term in Maxwell’s equations. It takes two objects. First a field functor — this is the part that runs *on the GPU*, once per particle:

```
template <Dimension D>
struct ExtFields {
    ExtFields(real_t e1) : e1 { e1 } {}
    Inline auto ex1(const coord_t<D>&) const -> real_t { return e1; } // ignores x
private:
    const real_t e1;
};
```

and then the hook that hands one to the engine each step:

```
auto ExternalFields(simtime_t t, spidx_t, const Domain<S, M>&) const
-> std::pair<bool, ExtFields<M::Dim>> {
    return { A0 != ZERO, ExtFields<M::Dim>{ A0 * math::cos(omega * t) } };
}
```

The split matters for understanding what is cheap and what is not. `ExternalFields` is called **on the host**, once per species per timestep (`src/kernels/pushers/sr_policies.h`); the functor it returns is then evaluated **on the device**, once per particle. Because this drive is uniform in space, all of the time dependence lives in the host-side scalar and the per-particle work is a single register read — the cosine is evaluated a few times per step, not 10^8 times. The `bool` is a cheap switch: return `false` and the engine compiles the external-field branch out of the push entirely.

How the engine knows the hook is there. It is not registered, inherited or declared anywhere. `ENTITY` tests the generator against C++20 concepts in `src/global/traits/pgen.h` — `HasExternalFields` asks whether a method of that name and shape exists — and compiles the call in if it does, or a no-op if it does not. So every hook is opt-in by simply writing it, and omitting one is not an error. The flip side is that a method with a slightly wrong signature is not a compile error either: it is silently *not a hook*, and the feature just does not happen. The run log prints which hooks were detected (`engines/reporter.h`), which is the quickest way to confirm one took effect.

9.6 Step 6: report the drive (optional)

```
auto CustomStat(const std::string& name, timestep_t, simtime_t t,
                const Domain<S, M>&) const -> real_t {
    if (name == "e_ext") { return ext_elfield(t); }
    return ZERO;
}
```

Named entries requested under `[diagnostics]` are routed here and land as extra columns in the statistics CSV, alongside the field and particle energies. This is how the drive’s amplitude and phase become directly comparable with the plasma response in the same file rather than having to be reconstructed afterwards. It is optional, and nothing else depends on it.

9.7 The minimal generator, in full

Steps 2–5 assembled. This is a complete, working resonance setup:

```
namespace user {
  using namespace ntt;
  using prmvec_t = std::vector<real_t>;

  template <Dimension D>
  struct ExtFields {
    ExtFields(real_t e1) : e1 { e1 } {}
    Inline auto ex1(const coord_t<D>&) const -> real_t { return e1; }
  private:
    const real_t e1;
  };

  template <SimEngine::type S, class M>
  struct PGen {
    static constexpr auto D { M::Dim };
    static constexpr auto engines = traits::pgen::compatible_with<SimEngine::SRPIC>{};
    static constexpr auto metrics = traits::pgen::compatible_with<Metric::Minkowski>{};
    static constexpr auto dimensions = traits::pgen::compatible_with<Dim::_1D>{};

    const SimulationParams& params;
    const real_t A0, omega;
    const prmvec_t densities, temperatures;

    PGen(const SimulationParams& p, const Metadomain<S, M>&)
      : params { p }
      , A0 { p.template get<real_t>("setup.A0", ZERO) }
      , omega { p.template get<real_t>("setup.omega", ONE) }
      , densities { p.template get<prmvec_t>("setup.densities", prmvec_t{TWO}) }
      , temperatures { p.template get<prmvec_t>("setup.temperatures", prmvec_t{ZERO,ZERO}) } {}

    void InitPrtls(Domain<S, M>& domain) {
      arch::InjectUniformMaxwellians<S, M>(params, domain, densities[0],
      { temperatures[0], temperatures[1] }, { 1, 2 });
    }

    auto ExternalFields(simtime_t t, spidx_t, const Domain<S, M>&) const
      -> std::pair<bool, ExtFields<M::Dim>> {
      return { A0 != ZERO, ExtFields<M::Dim>{ A0 * math::cos(omega * t) } };
    }
  };
}
```

Roughly forty lines, of which two are physics. Everything in the production file beyond this is either a loader (§11) or a second drive family (§10).

9.8 Step 7: a deck, a build, a run

The deck needs the standard ENTITY sections — [simulation], [grid], [scales], [algorithms], [particles], [diagnostics], [output], covered in §4 — plus the [setup] block this generator just defined:

```
[setup]
densities      = [2.0]
temperatures   = [1.0e-3, 1.0e-3]
A0             = 9.48683e-4
omega          = 1.0
```

Then configure, compile and run. §12 is the full recipe with the three traps that make the configure line longer than it looks; the short form on a login node is:

```

module load gcc/13.2.0
cmake -B build-dpdm -D pgen=dpdm -D precision=double \
      -D deposit=esirkepov -D shape_order=5 \
      -D Kokkos_ENABLE_OPENMP=ON -D Kokkos_ENABLE_SERIAL=OFF \
      -D Kokkos_ARCH_ZEN3=ON -D CMAKE_IGNORE_PATH=$PWD/cmake
cmake --build build-dpdm -j
grep -E "^Kokkos_ENABLE_(OPENMP|SERIAL|CUDA):BOOL" build-dpdm/CMakeCache.txt
./build-dpdm/src/entity.xc -input resonance.toml

```

`-D pgen=dpdm` is not optional: the default is `."`, and with it CMake configures the libraries but never adds the executable target, so the build “succeeds” and produces no `entity.xc`. The `grep` is the verification step from §12 — a Serial build is indistinguishable from the outside and runs $\sim 112\times$ slower.

9.9 Summary of the interface

Hook	Detected by	Role here
<i>(constructor)</i>	required	reads <code>[setup]</code> , validates it
<code>engines, metrics, dimensions</code>	required	declares what the deck may ask for
<code>InitPrtls</code>	<code>HasInitPrtls</code>	the $t = 0$ particle state
<code>ExternalFields</code>	<code>HasExternalFields</code>	the dark-photon drive
<code>CustomStat</code>	<code>HasCustomStatOutput</code>	writes <code>e_ext</code> to the CSV

Table 2: Everything below the first two rows is opt-in: a generator that defines none of them compiles and runs, it simply has no particles and no extra physics. Concepts are in `src/global/traits/pgen.h`.

What the production generator adds on top of the minimum, and where each is treated: the deterministic loader and its two variants (§11); the swept Landau–Zener phase (§10) and the two conventions for it (§17.1); the 1V option, which zeroes the two transverse velocity components to match SHARP; and the `e_ext` diagnostic.

On naming options rather than numbering them. Two of the four `[setup]` switches are, strictly, redundant. `drive = "resonance"` is identical to `"landau_zener"` with $\dot{\omega} = 0$, since $\phi = \omega_0 t + \frac{1}{2}\dot{\omega}t^2$ collapses to $\omega_0 t$; and `sweep_phase = "naive"` is *exactly* the integrated form at twice the sweep rate, since $(\omega_0 + \dot{\omega})t = \omega_0 t + \frac{1}{2}(2\dot{\omega})t^2$. The whole drive could therefore be three numbers — $A_0, \omega_0, \dot{\omega}$ — with no strings and no validation branches at all. The names are kept because a deck reading `omega_dt = 4.0e-5` records what was run but not *why*, and for a reproduction the intent is part of the result; the strings also propagate into output paths and run names. It is a legibility choice, not a necessity, and a generator written for a single case should probably not make it.

10 Problem generator

All problem-specific code lives in `pgens/dpdm/pgen.hpp`. It provides three things.

External field. An `ExternalFields` hook returning a uniform e_{x1} , applied to *both* species; the pusher scales by q/m per species, so the ions automatically receive $1/1836$ of the electron kick. Because the drive is uniform in space, the cosine is evaluated once per species per step on the host and only the scalar is handed to the kernel — not once per particle.

Two drive families. Selected by `setup.drive`:

$$\text{resonance: } \phi(t) = \omega t, \quad \omega = \omega_p \text{ fixed}, \quad (1)$$

$$\text{Landau-Zener: } \phi(t) = \omega_0 t + \frac{1}{2} \dot{\omega} t^2. \quad (2)$$

The swept phase is integrated *analytically*. Sampling $\omega(t)$ per step and multiplying by t accumulates a spurious chirp: the naive form deviates by up to $0.86 A_0$ over the sweep. The analytic form was verified against the closed expression to 2.8×10^{-14} (double precision, i.e. exact).

A custom diagnostic. `CustomStat` exposes `e_ext`, the instantaneous drive, so amplitude and phase can be checked against the plasma response in the same CSV. Note that `CustomStat` receives time one timestep behind the `time` column in the same row (`engine.hpp` passes `time - dt` to the writer); this is a diagnostic bookkeeping offset only — the pusher’s centring is correct, as the drive-normalisation measurement in §15 confirms.

11 Deterministic (“quiet”) particle loading

The paper initialises “all particles equally spaced, with ions and electrons colocated throughout the computational domain,” which ensures $\epsilon_{\text{noise}} = 0$ at $t = 0$. ENTITY’s stock injector places every particle at a uniformly random position in the domain — not even one per cell — so cell occupancy is Poisson and $\epsilon_{\text{noise}}(0) \neq 0$. A loader was therefore written for this work.

11.1 What the initial condition actually requires

Two things, and the second is easy to miss.

1. $\epsilon_{\text{noise}} = 0$ at $t = 0$. Stated explicitly in the paper.
2. **Prompt relaxation to the Poisson floor.** The paper’s Fig. A00 shows undriven runs starting at zero and rising to the analytic level $L^2/(12N_p N_x)$ within a few ω_p^{-1} , then sitting flat for 2×10^4 . The text says the same from the driven side: “it takes approximately $t\omega_p \sim 3\text{--}4$ for the driven energy to be converted into de-coherent (high- k) noise.”

The second requirement is not decorative. The paper’s entire N_{pc} criterion ($\bar{\epsilon}_{\text{noise}} \sim 2.4$ resonant, ≤ 0.02 swept) presumes that shot noise is present and sets the seed from which daughter modes grow. A loader that satisfies only the first produces a plasma with *no* seed at all.

11.2 Three noise channels

1. **Charge density.** $\rho(x, 0) \neq 0$ seeds $E(x, 0) \neq 0$. Killed by co-locating each electron with an ion. ENTITY’s random start already does this, so this channel was never the problem.
2. **Total density.** $n_{\text{tot}} = n_e + n_i$ fluctuates at $\sqrt{2/N_{\text{pc}}}$ even with perfect co-location. Since $\omega_p \propto \sqrt{n}$ this is a spread in the local resonant frequency — the channel that physically matters.
3. **Velocity space.** Randomly sampled velocities produce current fluctuations that regenerate E within a plasma period.

Channel 2 must be *zero* at $t = 0$ and must *reappear at the Poisson level within a few ω_p^{-1}* .

11.3 Construction

Positions form an exact lattice — cell i holds N_{pc} particles at sub-cell offsets $(s + \frac{1}{2})/N_{\text{pc}}$ — and ions reuse them exactly, so both channel 1 and channel 2 vanish to round-off at $t = 0$. Velocities are drawn pseudo-randomly, keyed on a `splitmix64` hash of the *global* cell index and slot, which keeps the loading independent of how MPI decomposes the domain.

```

i   = p / ppc           // cell
s   = p % ppc           // slot within the cell
phi = (s + 1/2)/ppc     // exact lattice: eps_noise(0) = 0
x   = xi_min + i + phi
key = (global cell)*1000003 + s // decomposition independent
u_e = sig1 * probit(hash_u01(4*key))
u_i = sig2 * probit(hash_u01(4*key+1))

```

Why the velocity assignment must be irregular. Two constructions that look reasonable both fail, for the same reason.

Velocity keyed on the class. Give the same velocity to the same slot in every cell. Each velocity class is then a comb of exactly one particle per cell, spacing Δx , all moving at one speed. A uniform comb translates rigidly and deposits exactly uniform density at any offset, because the shape function is a partition of unity at that spacing. The sum of such combs is uniform forever: the lattice **never decoheres**.

A rigid per-cell rotation. Shifting the assignment by iR in cell i does not help — each class is still a uniform comb, now of spacing $(1 - R/N_{\text{pc}})\Delta x$. Still rigid, still no clumping.

The assignment has to be irregular *from cell to cell*, which is what the hash provides. A worked example with $N_x = 4$, $N_{\text{pc}} = 4$ is the quickest way to see it: with class-keyed velocities, class A sits at $x = 0.125, 1.125, 2.125, 3.125$ — spacing exactly 1, a perfect comb. With hashed velocities it sits at $x = 0.375, 1.125, 2.875, 3.625$ — spacings 0.75, 1.75, 0.75, irregular, and it clumps as soon as it streams. Both have identical, perfectly flat density at $t = 0$.

Velocity normalisation, and why it is now skipped. An ordered stratified set $\{\text{probit}((j + \frac{1}{2})/N_{\text{pc}})\}$ has mean zero but variance below unity (-1.97% at $N_{\text{pc}} = 64$, -0.64% at 200), so the legacy loader measures the realised variance and divides it out. The sampling loader must *not* do this: its realised variance carries a physical $1/\sqrt{N}$ fluctuation, the velocity-space counterpart of the shot noise being restored, and normalising it away would re-quiet the channel the loader exists to open.

11.4 Validation: the loader gate

Three undriven runs ($A_0 = 0$, $N_{\text{pc}} = 2700$, identical decks) differing only in `setup.loading`. Analytic floor $L^2/(12N_p N_x) = 4.938 \times 10^{-8}$; the table gives ϵ_E as a fraction of it.

loading	$\omega_p t = 1$	10	50	200	500
quiet (sampling)	1.08	1.33	1.27	1.45	1.34
quiet_lattice (legacy)	5.1×10^{-14}	2.1×10^{-12}	1.7×10^{-11}	2.1×10^{-10}	3.4×10^{-10}
random (stock)	1.18	1.54	1.53	1.44	1.58

Table 3: Loader gate, job 3447505. `quiet` is at the Poisson floor by $\omega_p t = 1$ and flat — Fig. A00. The legacy loader is 10–15 orders below it and still climbing at $\omega_p t = 500$. `random` is the control: it lands on the same floor, confirming the formula and the N_p convention independently of any quiet-start logic. Script: `run_gate_loader.sh`.

Two traps. ENTITY’s first output row is **step 1**, not $t = 0$, so no dump shows the initial condition directly; $t = 0$ uniformity is exact by construction, and `quiet_lattice` measuring $\delta n/n = 0.000$ with $\max|\rho_q| = 6 \times 10^{-17}$ at step 1 confirms the deposition is exact for that position set. And $\epsilon_E(0) \approx 0$ for *all three* loadings, including `random`, because ENTITY initialises E to zero identically — the discriminator is the total density, never the field.

11.5 Consequences of the legacy loader

The legacy loader was used for the first round of production runs, and their saturation times and peak energies are not reproductions. Measured $k \neq 0$ field power relative to the $k = 0$ driven mode at $\omega_p t \approx 40$:

Run	$P_{k \neq 0}/P_{k=0}$	$\omega_p t$ at which the ratio reaches 1%
fast, legacy	2.7×10^{-31}	962
fast, sampling	1.0×10^{-2}	0
lz1p0, legacy	1.7×10^{-29}	1120
lz1p0, sampling	1.1×10^{-3}	0

2.7×10^{-31} is double-precision epsilon *squared*: the daughter modes were growing out of round-off. At $\gamma = 0.061 \omega_p$, climbing from 10^{-31} to order unity takes $\sim 1100 \omega_p^{-1}$ of extra runway, during which the drive keeps pumping.

The effect is large where saturation time sets the amplitude, and small otherwise. In the resonant cases the $k = 0$ wave grows as t^2 until daughters halt it, so a delay translates directly into a larger peak. In the swept case the energy available is set by how long ω lingers near ω_p , not by the onset of turbulence, so the same delay changes little:

Run	peak ϵ_E/ϵ_{th}	$\omega_p t$ at peak	$kT_e(\text{end})/kT_e(0)$
fast legacy $\rightarrow$ sampling	43.55 $\rightarrow$ 9.14	553 $\rightarrow$ 238	70.7 $\rightarrow$ 75.5
slow legacy $\rightarrow$ sampling	2.13 $\rightarrow$ 0.53	3694 $\rightarrow$ 1598	3.43 $\rightarrow$ 2.53
lz1p0 legacy $\rightarrow$ sampling	190.4 $\rightarrow$ 237.6	3689 $\rightarrow$ 4269	1003 $\rightarrow$ 962

A quiet start still does not permit a reduction in N_{pc} . With the sampling loader N_{pc} sets the seed amplitude directly, which is what the paper’s sizing assumes; under the legacy loader round-off set it and N_{pc} was doing nothing for the physics. Either way, particle counts must be sized from the shot-noise criterion (§7).

12 Building on Lonestar6

This section is the full recipe: the four build directories used in this work can be reconstructed from scratch by following it in order. The directories themselves are not version-controlled — they total ~ 1.2 GB of object files, fetched dependency sources and architecture-specific binaries, and are valid only on this machine.

12.1 Environment and prerequisites

All four builds are configured and compiled **on a login node**. ENTITY fetches Kokkos and ADIOS2 with CMake’s `FetchContent` at configure time, and LS6 compute nodes have no outbound network: the same commands on a compute node fail in the clone step.

```
module load gcc/13.2.0    # switches the hierarchy; brings impi/21.12 with it
module load cuda/12.8    # CUDA builds only -- LS6 defaults to cuda/11.4
```

gcc/13.2.0 is not optional. The binaries link against GCC 13’s `libstdc++`; the default LS6 environment provides gcc/9.4.0, under which the executable dies at load time with `GLIBCXX.3.4.31 not found before` it reads its input file. The module must therefore also be loaded in every job script, not only at build time. Loading it switches the module hierarchy to `/opt/apps/gcc13.2/modulefiles` and pulls in `impi/21.12` (Intel oneAPI MPI 2021.12, `mpigxx`), which is the MPI the two parallel builds pick up. CUDA must be requested explicitly: the site default `cuda/11.4` is too old for the fetched Kokkos, and the builds here used `cuda/12.8` (`nvcc` at `/opt/apps/cuda/12.8/bin/nvcc`).

12.2 Configure

Four configurations are used, differing only in backend. All are configured from the repository root:

```
IGN=-DCMAKE_IGNORE_PATH=$PWD/cmake
COMMON="-D pgen=dpdm -D deposit=esirkepov -D shape_order=5 -D precision=double"
KOK="-D Kokkos_ENABLE_OPENMP=ON -D Kokkos_ENABLE_SERIAL=OFF"

# 1. CPU, OpenMP          -- validation, fast resonance case
cmake -B build-dpdm        $COMMON $KOK -D Kokkos_ARCH_ZEN3=ON $IGN
# 2. CPU + MPI            -- multi-node fallback
cmake -B build-dpdm-mpi    $COMMON $KOK -D Kokkos_ARCH_ZEN3=ON -D mpi=ON $IGN
# 3. CUDA, single GPU     -- the production path
cmake -B build-dpdm-cuda   $COMMON -D Kokkos_ENABLE_CUDA=ON \
    -D Kokkos_ARCH_AMPERE80=ON $IGN
# 4. CUDA + MPI, multi-GPU -- LZ 0.1, long-term stability
cmake -B build-dpdm-cuda-mpi $COMMON $KOK -D mpi=ON -D gpu_aware_mpi=OFF \
    -D Kokkos_ENABLE_CUDA=ON -D Kokkos_ARCH_AMPERE80=ON $IGN
```

Kokkos_ARCH.ZEN3 targets the AMD EPYC 7763 CPU nodes and Kokkos_ARCH.AMPERE80 the A100s; configuration 3 omits \$KOK because a CUDA backend does not need a threaded host backend for this workload. Each `cmake -B` re-fetches and re-builds its own private copy of Kokkos and ADIOS2 under `<build>/_deps`, which is why four directories cost ~1.2 GB rather than four small binaries.

Historical note. Configuration 1 was in fact first configured with `-D adios2_DIR=$PWD/build/_deps/adios2-build` reusing the ADIOS2 that an earlier unrelated `build/` directory had already fetched. That works, but it is not reproducible from a clean checkout — there is no `build/` to point at — so the form given above, using \$IGN like the other three, is the one to follow. The two are equivalent: both end up at ADIOS2 2.11.0.

12.3 Compile

```
cmake --build build-dpdm      -j 8
cmake --build build-dpdm-mpi  -j 8
cmake --build build-dpdm-cuda -j 8
cmake --build build-dpdm-cuda-mpi -j 8
```

Each produces its executable at `<build>/src/entity.xc`; that path is what the job scripts in `pgens/dpdm/` invoke. The CUDA configurations take substantially longer than the CPU ones — every kernel is compiled twice, to PTX and to Ampere SASS. Keep `-j` modest: LS6 login nodes are shared, and a large `-j` on an `nvcc` build is the usual way to get a job killed by the login-node watchdog.

Verify before trusting a build. Configure-time options fail silently here (Trap 2 below), so check the cache rather than assuming:

```
grep -E "Kokkos_ENABLE_(OPENMP|SERIAL|CUDA):BOOL" build-*/CMakeCache.txt
```

Trap 1: the ADIOS2 shim shadows its own package. `find_package(adios2 QUIET)` matches the repository's own `cmake/adios2Config.cmake`, which only sets cache variables and exports neither a version nor any targets. CMake nevertheless sets `adios2_FOUND` and the fetch branch never runs. The symptom is ADIOS2: `v` with a path pointing into `entity/cmake`, followed by Target "entity.xc" links to `adios2::cxx_mpi` but the target was not found. Excluding that directory via `CMAKE_IGNORE_PATH` restores the intended fetch. This is an upstream bug: the shim shadows the package it exists to configure.

Trap 2: CMAKE_IGNORE_PATH also disables Kokkos OpenMP. The same exclusion blocks the repository’s Kokkos settings file, and a freshly fetched Kokkos defaults to **Serial only**. A Serial build is externally indistinguishable from a threaded one and runs $\sim 112\times$ slower (NLWP = 2, 99.5% CPU). Neither SLURM’s `-c` default nor `ibrun` binding produces this symptom, so there is no point looking there; the cache check at the end of the previous subsection is mandatory rather than advisory.

Trap 3: ADIOS2 target rename. ADIOS2 2.11 renamed the C++ bindings target from `cxx11` to `cxx` (and `cxx11_mpi` to `cxx_mpi`). ENTITY hardcodes the 2.11 names, so TACC’s `adios2/2.10.2` module cannot link even though it does provide `ADIOS2_HAVE_MPI`. Fetching 2.11.0 (ENTITY’s pinned tag) is the clean route, and requires a login node — compute nodes have no outbound network.

GPU-aware MPI. Disabled (`gpu_aware_mpi=OFF`). It would require `mvapich2-gdr-cuda12`, a different MPI stack from the `impi` in the `gcc13` hierarchy. The halo here is a few ghost cells of three field components, so host staging is free at this problem size.

13 Measured performance

This section records the throughput measurements that the sizing procedure of §7 draws on.

13.1 Throughput

Configuration	ms/step at $N_{pc} = 2 \times 10^5$	Note
1 CPU node (128 cores)	~ 950	steady state; rises +48% from 640
8 CPU nodes, MPI	135	$4.94\times$ speedup, 69% efficiency
1× A100	$265 \rightarrow 516$	$3.6\times$ a full CPU node
3× A100 (one node)	128	$2.07\times$ over 1 GPU, 69% efficiency

Table 4: Measured on 4×10^8 particles, $N_x = 1000$.

A single A100 outperforms an entire 128-core node by $3.6\times$. The feared collapse from deposition atomics on a 1000-cell grid did not occur: going from $N_{pc} = 2 \times 10^4$ to 2×10^5 — a tenfold increase in particles per cell, and therefore in atomic contention per cell — cost only a factor 1.42 ($0.444 \rightarrow 0.630$ ns per particle-step).

13.2 Per-step cost doubles after saturation

This governs all walltime planning.

On a single A100 at $N_{pc} = 2 \times 10^5$ the cost is 265 ms/step while the plasma is laminar and 516 ms/step once it saturates and becomes turbulent — particles disorder, and the memory locality the early rate depended on is lost. A 2000-step benchmark ($\omega_p t = 40$) shows a dead-flat 265 ms, which is *not* evidence of stability: the run saturates at $\omega_p t \approx 2000$, so such a benchmark never leaves the laminar phase. Taken at face value it overran a 48 h wall at 88% completion on the slow resonance case, which then required a checkpoint restart.

Rule: benchmark across the physics transition, or budget a factor two.

Part III

Runs, settings, and comparison with the paper

14 Parameters taken from the paper source

All values below are quoted from `main.tex` in the arXiv source package, not inferred from the text or figures.

Quantity	Paper value	This work
Grid	$N_x = 1000, L = 40 c/\omega_p$	identical
Cell size	$\Delta x = 0.04 c/\omega_p$	identical
Debye length	$\lambda_D \approx 0.033 c/\omega_p$	identical
Temperatures	$k_B T_e = k_B T_i = 10^{-3} m_e c^2$	identical
Mass ratio	$m_i/m_e = 1836$	identical
Thermal speed	$v_{th,e}^2 \equiv \langle (v_x - \langle v_x \rangle)^2 \rangle = k_B T/m$	identical (1V)
Thermal energy density	$5 \times 10^{-4} n_e m_e c^2$	4.9963×10^{-4} measured
Deposition	5th-order splines	<code>esirkepov, shape_order=5</code>
Pusher	Vay (2008)	<code>Vay</code>
Timestep	$\Delta t \approx 0.02/\omega_p$	$CFL = 0.5 \Rightarrow 0.02$
Initial loading	equally spaced, co-located	§11
Shot noise	$\epsilon_{noise} = L^2/(12N_p N_x)$	agrees to $3.1 \times$
Resonance N_{pc}	2×10^2 and 2×10^5	identical
LZ sweep	$\omega(t) = 0.8\omega_p + 0.1\omega_p t/5000$	identical
LZ amplitudes	$v_q^D/v_{th}^e = 1.0$ and 0.1	identical
LZ N_{pc} criterion	$\bar{t}_{noise} \leq 0.02$	$2.7 \times 10^3, 2.7 \times 10^5$

Table 5: Paper parameters and the corresponding settings used here.

Two points deserve emphasis.

The 1V definition. The paper defines the thermal speed from the *one-dimensional* x -component variance, $v_{th,e}^2 = \langle (v_x - \langle v_x \rangle)^2 \rangle = k_B T/m$. ENTITY’s Maxwellian is isotropic 3V. Because there are no transverse forces in a 1D electrostatic run, the perpendicular degrees of freedom merely carry constant energy; including them would dilute T_e by a factor three. The quiet loader therefore sets $u_{x2} = u_{x3} = 0$ (`one_v = true`), matching SHARP exactly and removing the 3V-versus-1V correction from every downstream comparison. The paper’s quoted thermal energy density 5×10^{-4} matches the 1V value and is used as the reference line throughout.

The Landau–Zener crossing time. With $\omega_0 = 0.8$ and $\dot{\omega} = 2 \times 10^{-5}$, resonance ($\omega = \omega_p$) is crossed at $\omega_p t = 10^4$. The paper’s statement that “nonlinearity kills the resonant transfer around $\omega_p t = 5000$ ” refers to $\omega = 0.9\omega_p$, i.e. *before* the crossing — off-resonant excitation, not the crossing itself.

15 Preparatory and calibration runs

Before the production runs, a sequence of measurements pinned the unit conventions and validated the drive. These are summarised here because several production choices depend on them.

- **Units.** `skindepth0 = 1` with `densities = [2.0]` gives $n_e = n_0$ exactly, hence $\omega_{pe} = 1$. Confirmed by `T00.1` rest-mass part = 1.0000.
- **Energy conservation.** Over 2×10^4 steps, $\Delta T^{00} = -\frac{1}{2}\Delta\langle E^2 \rangle$ with $r = -0.996$; no grid heating.

- **ω_p calibration.** A five-point detuning scan ($\omega = 0.96\text{--}1.04$) is symmetric about $\omega = 1$ and follows the linear-theory sinc^2 envelope; the on-resonance case grows as t^2 across two decades in energy while every detuned case remains bounded.
- **Drive normalisation.** Least squares over $10 < \omega_p t < 40$ gives $C = 0.12432$ against Eq. A25’s $1/8 = 0.125$ (0.5%). The ion response ratio is 5.70×10^{-4} against $1/1836 = 5.45 \times 10^{-4}$, confirming the drive reaches both species with the correct q/m .

A factor-of-two trap. It is the *sum* $\epsilon_E + \Delta K E_e$ that obeys the t^2 law, not the field energy alone. The resonantly driven Langmuir wave equipartitions, so the field carries only half the wave energy on cycle average and swings between 3% and 85% of it instantaneously at $2\omega_p$. The paper’s linear-theory line must be compared against the sum.

16 The two resonance cases (paper Fig. 2)

Both run to $\omega_p t = 10^4$, 1V, with the sampling loader of §11.3. The legacy-loader values are shown alongside because they appeared in earlier drafts and are *not* reproductions (§11.5).

Case	loader	N_{pc}	A_0	peak $\epsilon_E/\epsilon_{\text{th}}$	$kT_e(\text{end})/kT_e(0)$
fast, $v_q^D/v_{\text{th}}^e = 3 \times 10^{-2}$	legacy	2×10^2	9.48683×10^{-4}	43.55 (at 553)	70.7
	sampling	2×10^2	9.48683×10^{-4}	9.14 (at 238)	75.5
slow, $v_q^D/v_{\text{th}}^e = 1 \times 10^{-3}$	legacy	2×10^5	3.16228×10^{-5}	2.13 (at 3694)	3.43
	sampling	2×10^5	3.16228×10^{-5}	0.53 (at 1598)	2.53

Table 6: Production resonance runs, legacy versus sampling loader. $\epsilon_{\text{th}} = 5 \times 10^{-4}$ is the 1V electron thermal energy density. Runs `fast_s` and `slow_s`; the legacy rows are `fast`, `slow`.

Both cases now agree with the paper. For the fast-growth regime the paper says the electron temperature grows to $O(10^2)$ of its initial value: we measure 75.5. For the slow-growth regime it says the transfer slows “*before* the energy density stored in the Langmuir wave becomes comparable to the initial thermal energy” and that “the total energy transfer is about the electron thermal energy”: we measure a field peak of $0.53 \epsilon_{\text{th}}$, which **never crosses the thermal line at any sample**, and a peak total transfer ($\epsilon_E + \Delta K E_e$) of $1.59 \epsilon_{\text{th}}$.

The regime distinction. The paper separates slow and fast growth at $v_q^D/v_{\text{th}}^e \sim (m_e/m_p)^{1/2}/2 \approx 0.0117$. The discriminator is the lasting electron heating — 75.5 against 2.53, a factor ~ 30 — not the peak field energy. With the sampling loader the two cases also separate cleanly on whether the field energy reaches the thermal line at all: the fast case exceeds it by $9\times$, the slow case never reaches it.

Initial temperature. $k_B T_e(0) = 4.9963 \times 10^{-4}$ against the 1V target 5.0×10^{-4} . The -0.075% residual is *not* loading error but the relativistic definition $T^{00} = \sum w m \gamma$, whose next term is $-\frac{3}{8}\langle u^4 \rangle = -0.075\%$. The ions, 1836 times heavier and far more non-relativistic, land at 4.999998×10^{-4} , which confirms the interpretation.

Cost. On the legacy loader the slow case took ~ 51 GPU-hours on one A100 (48 h to 88%, then a 14.5 h restart). The sampling re-run used three A100s on one node and took **43 h** against an estimate of ~ 25 h from the measured $2.07\times$ scaling: saturation arrives earlier with a physical seed, so more of the run sits in the post-saturation regime where the per-step cost roughly doubles (§13.2). Estimates built on the laminar rate are optimistic by about a factor two even after that effect is known about.

17 Landau–Zener runs

The Landau–Zener family is the physically realistic case: in cosmology it is $\omega_p(t)$ that drifts through the dark-photon mass. The paper instead sweeps $m_{A'}(t)$ at a numerically tractable rate, quoting $\omega_p/H \approx 7 \times 10^4$.

17.1 The sweep convention

The paper’s appendix gives the drive as $A_0 \cos(\omega t)$ with $\omega(t) = 0.8\omega_p + 0.1\omega_p t/5000$. That admits two readings, and they are not equivalent:

setup.sweep_phase	phase	instantaneous frequency	crossing
integrated (default)	$\omega_0 t + \frac{1}{2}\dot{\omega}t^2$	$\omega_0 + \dot{\omega}t$	$\omega_p t = 10^4$
naive	$(\omega_0 + \dot{\omega}t)t$	$\omega_0 + 2\dot{\omega}t$	$\omega_p t = 5000$

`integrated` is the correct realisation of “a drive whose frequency evolves as $\omega(t)$ ” and is the default. `naive` is a well-defined chirp sweeping at twice the nominal rate, provided so the paper’s convention can be tested.

The paper’s own evidence is contradictory. Its stated $\omega_p/H \approx 7 \times 10^4$ matches `integrated`: with $d \ln \omega_p^2/dt = 3H$ and $H = 2\dot{\omega}/(3\omega)$ at crossing, $\dot{\omega} = 2 \times 10^{-5}$ gives 7.5×10^4 , against 3.75×10^4 for the naive form. But its *figure* points the other way: the x -axis runs 0 to 10^4 and the linear-theory curve completes its rise and goes flat by $\omega_p t \approx 5300$. Linear Landau–Zener theory transfers at the crossing, so the paper’s crossing is near 5000 — where `naive` crosses, and where `integrated` has nothing happening. The figure is the stronger evidence, which makes the quoted ω_p/H too large by a factor two.

17.2 Results, and an unresolved discrepancy

Run	loader / sweep	crossing	peak $\epsilon_E/\epsilon_{\text{th}}$	$kT_e(\text{end})/kT_e(0)$
1z1p0	legacy / integrated	10^4	190.4 (at 3689)	1003
1z1p0.s	sampling / integrated	10^4	237.6 (at 4269)	962
1z1p0n	sampling / naive	5000	246.7 (at 2170)	826
paper (digitised)	—	≈ 5000	—	≈ 1.3

Table 7: Landau–Zener at $v_q^D/v_{\text{th}}^e = 1$, $N_{\text{pc}} = 2.7 \times 10^3$, $\omega_p t = 2 \times 10^4$. The paper’s value is read off its figure: the red PIC curve moves 17 pixels over the full x -range at ~ 153 px per decade, i.e. a factor $10^{0.11} = 1.3$, which is its “the kinetic energy of the electrons is not even changed by a factor of 2”.

We measure $\sim 10^3$ where the paper reports $\lesssim 2$, and the cause is not yet identified. Four candidates have been tested and excluded:

1. **The loader** (§11.5). $1003 \rightarrow 962$, -4% . Predicted in advance from `fast`, where the loader moved the peak field energy $4.8\times$ but ΔKE_e only 7% .
2. **The sweep convention** (§17.1). $962 \rightarrow 826$, -14% . The convention demonstrably took effect — the peak moved from $\omega_p t = 4269$ to 2170, halving as a doubled sweep rate requires — so this is a real null, not a failed switch.
3. **Coherent versus random energy**. T^{00} counts the uniform $k = 0$ quiver motion, which is sloshing rather than heating, and at $v_q^D/v_{\text{th}}^e = 1$ the forced oscillation alone is $2.78 v_{\text{th}}^e$. Decomposing into $\frac{1}{2}\langle u \rangle^2$ and $\frac{1}{2}(\langle u^2 \rangle - \langle u \rangle^2)$ gives a coherent fraction never above 0.5% ; the random ratio equals the total ratio to three digits. The heating is genuinely thermal.

4. **The drive implementation.** `e_ext` matches $A_0 \cos(\omega_0 t + \frac{1}{2}\dot{\omega}t^2)$ to 9.5×10^{-15} once the `CustomStat` $t - \Delta t$ offset is applied, and does not match the naive form at all. Amplitude, sweep rate and waveform are all as intended.

Against this, both resonance cases now agree with the paper (§16), so the machinery is sound and the disagreement is specific to the swept drive. One diagnostic: by the end of the run $u_{\text{rms}} \approx 1.2c$, so our electrons leave the non-relativistic regime the paper works in — a symptom of the excess energy rather than an explanation for it.

17.3 Particle counts, and why the low-amplitude case stays gated

Before resonance the swept drive is a *bounded* forced oscillation of amplitude $A_0/(1 - \omega^2/\omega_p^2) = 2.78A_0$ at $\omega = 0.8\omega_p$, not a growing one, so there is no time at which the signal outgrows the noise. The paper requires $\bar{t}_{\text{noise}} = L/(A_0\sqrt{3N_p N_x/2}) \leq 0.02$, giving $N_{\text{pc}} = 2.7 \times 10^3$ and 2.7×10^5 for the two amplitudes. Verified for the run used here: $N_{\text{pc}} = 2700$ gives $\bar{t}_{\text{noise}} = 0.0199$.

The $v_q^D/v_{\text{th}}^e = 0.1$ case is written (`run_lz0p1.sh`, $N_{\text{pc}} = 2.7 \times 10^5$, 5.3×10^8 particles ≈ 31 GB, three A100s, ~ 81 h chained) and deliberately **not** run. It would cost ~ 243 GPU-hours to produce a second number against the same unresolved normalisation.

18 Superseded results

Figures from earlier drafts of this document, and from comments still present in some older scripts, that are wrong. Listed so they are recognisable if encountered — not as a history.

Superseded claim	Correct statement
“The quiet start is validated: $\delta n/n = 0$ exactly, noise floor 1.31×10^{-17} against 3.03×10^{-6} random.”	Those gates measured the wrong property. The paper’s initial condition requires $\epsilon_{\text{noise}}(0) = 0$ <i>and</i> relaxation to the Poisson floor within a few ω_p^{-1} (its Fig. A00). The legacy loader satisfied only the first: keying velocity on the class makes every class a rigid comb, so the lattice never decohered and driven runs seeded daughter modes from round-off (§11.5). “Noise floor 10^{-17} ” was the defect, reported as an achievement.
“The failed first design gave $\delta n/n = 6.3 \times 10^{-3}$ after one step.”	That is $1/\sqrt{N_{\text{pc}}}$ — physical shot noise, and the paper’s behaviour. It was engineered away.
“Both resonance cases overshoot the thermal energy in the field energy (43.6 versus 2.1).”	An artefact of the legacy loader. With the sampling loader the fast case peaks at $9.14\epsilon_{\text{th}}$ and the slow case at 0.53, never crossing the thermal line — which is what the paper describes (§16).
“The fast case should peak near $\omega_p t \approx 110$, as seen in earlier random-start scans.”	110 is the analytic estimate $t_{\text{sat}} + 1/\omega_{pi}$, not a measurement, and was mislabelled as observed. The random-start scan (<code>run34 r3e-2</code>) peaks at $\omega_p t = 241$. The sampling loader gives 238 — agreement with its control to 1%.
“The paper’s $\omega_p/H \approx 7 \times 10^4$ shows it used the integrated sweep phase.”	Its figure places the crossing near $\omega_p t = 5000$, which is the naive convention; the quoted ω_p/H is then too large by two (§17.1). This argument was used to cancel the naive-sweep test before it ran.
“The shot-noise floor is $650\times$ below Eq. B4.”	A units error: N_p is the <i>total</i> macroparticle count, not per cell. Correctly evaluated, Eq. B4 agrees with measurement at a flat $3.1\times$ and is the right tool for sizing (§7).
“Per-step cost is stable.”	It roughly doubles at saturation (§13.2). A benchmark confined to the laminar phase measures the wrong number; the slow re-run took 43 h against a 25 h estimate for the same reason.

Two recurring failure modes are worth naming. The first is extrapolating a measurement outside the regime where it was taken: the cost model, calibrated at $N_{\text{pc}} \leq 8000$, predicted 8 minutes for a run that took 41 and 97 seconds for one that exceeded 40. The second is **validating against a property that is easy to measure rather than the one that matters** — the quiet-start gates checked that the lattice was perfect, when the requirement was that it stop being perfect promptly. Both survive a long time because the wrong quantity looks excellent.

19 Status and remaining work

Complete. The quiet-start loader and its gate against the paper’s Fig. A00; both resonance cases of Fig. 2 with the corrected loader, *both now agreeing with the paper*; the Landau–Zener case at $v_q^D/v_{\text{th}}^e = 1$ under both sweep conventions; MPI decomposition independence; the full performance characterisation; and the reduction of every completed run to `.npz` (`extract_paper.py`, ~4–21 min on one `vm-small` node, 50 MB).

Run	tag	wall	outcome
loader gate	<code>gate_*</code>	20 min	at the Poisson floor by $\omega_p t = 1$
resonant fast	<code>fast_s</code>	31 min	$kT_e \times 75.5$, paper says $O(10^2)$
resonant slow	<code>slow_s</code>	43 h	peak $0.53 \epsilon_{\text{th}}$, never crosses thermal
LZ, integrated sweep	<code>lz1p0.s</code>	2:46	$kT_e \times 962$
LZ, naive sweep	<code>lz1p0n</code>	3:03	$kT_e \times 826$

Blocking. The Landau–Zener heating discrepancy: $\sim 10^3$ against the paper’s ≈ 1.3 (§17). Four candidates tested and excluded — loader, sweep convention, coherent/random decomposition, drive implementation. Since both resonance cases now reproduce, the fault is specific to the swept drive rather than to the code generally. The $v_q^D/v_{\text{th}}^e = 0.1$ case stays gated: ~243 GPU-hours for a second number against the same unresolved normalisation.

Open.

- **The heat gate predates the loader fix.** The numerical-heating bound over 10^6 steps ($\Delta T_e = -0.016\%$) was measured with the legacy loader, whose lattice cannot heat because it cannot decohere. It needs re-running (~2 h) before the Landau–Zener results rest on it.
- **Only the Landau–Zener figure has been digitised.** The resonance comparison still rests on the paper’s text. The published Fig. 2 would give peak $\epsilon_E/\epsilon_{\text{th}}$ and its timing directly, which are the sharpest discriminators available.
- **Long-term stability** ($\omega_p t \rightarrow 8 \times 10^4$), the paper’s strongest claim, is untested; at $N_{\text{pc}} = 2 \times 10^5$ it is roughly eight times the slow case, which already took 43 h on three A100s.
- **2D cross-check** not run; 1+1D sufficiency is assumed on the paper’s word.
- **Collisions.** ENTITY, like SHARP, is collisionless. “Persists indefinitely” should be read as “on collisionless timescales”.